

COIT12209 – Data Science.

Assessment 2 – Explanatory Data Analysis Report.

By Kseniia Zaichenko

Table of Contents

Introduction	3
Subtask 1: Explanatory Data Analysis	3
Original Dataset Overview	3
Data Cleaning and Handling Data-Related Issues.....	7
Descriptive Summary of the Cleaned Dataset	12
Subtask 2: Prediction Model Discussion	19
Model preparation and 70/30 Train-Test Split.....	19
Regression Models Development	21
Feature importance.....	22
Model Evaluation and Metric Justification	24
Accuracy and Interpretability Trade-Off	26
Conclusion and Recommendations	26
Main Findings	26
Limitations, Biases, and Overfitting Risks	26
Recommendations for future improvement	26
References	27

Introduction

With urban development and growing concern about pollution, bicycles are becoming an increasingly popular, eco-friendly and convenient mode of transport in many Australian cities. As bike rental systems grow in popularity, they generate large volumes of data that can be used to analyse demand and inform data-driven decisions. Thus, this report analyses a dataset provided by the Rockhampton Regional Council containing daily bike rental records, with the objective of identifying factors influencing demand and investigating seasonal and weather patterns in service usage. The council also wants to assess the feasibility of demand forecasting for future planning, such as expanding the number of stations in busy areas or increasing bike availability during peak seasons.

Subtask 1: Explanatory Data Analysis

Original Dataset Overview

The dataset ('bike_rental.csv'), provided by Rockhampton Regional Council, contains daily bike rental records with a mix of temporal, categorical, binary, and numerical variables. The only temporal variable is 'date', while the categorical variables include 'season' and 'weather'. The binary variables, 'holiday', and 'workingday', are coded as 0 or 1. The numerical variables include 'temp', 'atemp', 'humidity', 'windspeed' and 'count'. In this analysis, 'count' was treated as the target variable because it represents the total number of bike rentals to be analysed and predicted. The remaining variables were treated as potential predictors because they describe temporal, weather-related and calendar-based factors that may influence daily bike rental demand.

The original dataset overview was used to assess its structure before cleaning. The initial inspection output showed that the original dataset contained 761 rows and 10 columns before cleaning and confirmed that the structure was not damaged, with no missing columns, as shown in Figure 1. Additionally, the output identified missing values and duplicate rows, as illustrated in Figure 2.

```
Original dataset shape: (761, 10)

Column names:
Index(['date', 'season', 'holiday', 'workingday', 'weather', 'temp', 'atemp',
       'humidity', 'windspeed', 'count'],
      dtype='str')

Data types:
date           str
season         str
holiday        int64
workingday     int64
weather        str
temp          float64
atemp         float64
humidity       float64
windspeed     float64
count         float64
dtype: object
```

Figure 1. Code output for initial dataset structure check (shape, column names and data types).

```

Missing values:
date          0
season        0
holiday       0
workingday    0
weather       0
temp         26
atemp        26
humidity      26
windspeed    26
count        28
dtype: int64

Duplicate rows:
23

```

Figure 2. Code output for initial data quality check (missing values and duplicate rows).

Following the initial dataset overview, additional inspection checks were conducted before cleaning to identify data quality issues beyond missing variables and duplicate rows. These checks were guided by the validation rules summarised in Table 1. The acceptable ranges and values, as well as validation rules, were developed based on information provided by the Council and external research about the Rockhampton Region. Furthermore, Table 1 includes an interquartile range (IQR) based diagnostic check as a validation rule for the ‘count’ variable, identifying extreme values based on the dataset's distribution rather than a fixed upper limit, since the Council did not provide an official maximum theoretical rental ‘count’. IQR-based diagnostics were added because unusually high ‘count’ values could distort visual analysis and compromise subsequent model training, which would be crucial given that ‘count’ is the target variable in the dataset.

Table 1. Acceptable range and values for the bike_rental dataset's variables.

Variable	Acceptable Range or Values	Reason for the Validation Rule
‘date’	Valid calendar date	Dates were required to be valid calendar dates, taking leap-year rules into account.
‘season’	‘Summer’, ‘Autumn’, ‘Winter’, ‘Spring’	‘season’ is a categorical variable and should only contain ‘Summer’, ‘Autumn’, ‘Winter’, ‘Spring’.
‘weather’	‘Clear’, ‘Cloudy’, ‘Rain’, ‘Storm’	‘weather’ is a categorical variable and should only contain ‘Clear’, ‘Cloudy’, ‘Rain’, ‘Storm’.
‘holiday’	0 or 1	‘holiday’ is a binary-coded variable, so only 0 or 1 were considered valid.
‘workingday’	0 or 1	‘workingday’ is a binary-coded variable, so only 0 or 1 were considered valid.
‘temp’	-2°C to 46°C	‘temp’ values outside this range were treated as unrealistic for this dataset, given the Rockhampton region, where the lowest recorded annual temperature is

		approximately -1.0°C and the highest is 45.3°C (Current Results n.d.; Bureau of Meteorology n.d.).
‘atemp’	-5°C to 50°C	‘atemp’ values outside this range were treated as unrealistic for the dataset's context, since apparent temperature typically spans a slightly wider range than actual temperature.
‘humidity’	0% to 100%	‘humidity’ is a percentage that ranges from 0 to 100%.
‘windspeed’	0 to 100 km/h	‘windspeed’ cannot be negative, and values above 100 km/h were treated as unrealistic for this dataset.
‘count’	≥ 0; extreme values above the IQR	The rental count cannot be negative or exceed the IQR boundaries.

After defining the validation rules, the categorical and binary columns were checked against their expected values, as shown in Figure 3. The code stored the accepted values for season, weather, holiday and working day, then compared each column in the original dataset against those accepted values. This check identified two inconsistent category labels: “Summr” in the ‘season’ column and “Rainn” in the ‘weather’ column, and no unexpected values were found in the binary columns ‘holiday’ and ‘workingday’, as shown in Figure 4. The inconsistent labels were treated as spelling errors and corrected during the data cleaning stage to prevent them from being interpreted as separate categories in future visual analysis or modelling.

```
# Validate categorical and binary columns against expected values

expected_values = {
    "season": ["Summer", "Autumn", "Winter", "Spring"],
    "weather": ["Clear", "Cloudy", "Rain", "Storm"],
    "holiday": [0, 1],
    "workingday": [0, 1]
}

validation_results = []

# Identification of spelling mistakes or unexpected values before cleaning
for column, valid_values in expected_values.items():
    unexpected_values = bike_data.loc[
        ~bike_data[column].isin(valid_values),
        column
    ].dropna().unique()

    unexpected_values = sorted(list(unexpected_values), key=str)

    validation_results.append({
        "Column": column,
        "Expected values": ", ".join(map(str, valid_values)),
        "Unexpected values": ", ".join(map(str, unexpected_values)) if unexpected_values else "None",
        "Number of unexpected rows": (~bike_data[column].isin(valid_values)).sum()
    })

validation_results = pd.DataFrame(validation_results)

display(validation_results)
```

Figure 3. Code to validate categorical and binary values against expected values before cleaning.

	Column	Expected values	Unexpected values	Number of unexpected rows
0	season	Summer, Autumn, Winter, Spring	Summr	1
1	weather	Clear, Cloudy, Rain, Storm	Rainn	1
2	holiday	0, 1	None	0
3	workingday	0, 1	None	0

Figure 4. Output of categorical and binary validation against expected values.

After validating the categorical variables, the numerical variables were also checked for compliance with the validation rules defined earlier in Table 1. This check included a descriptive summary of the original numerical variables and diagnostic counts of values outside acceptable ranges. To distinguish between clearly incorrect values and theoretically possible values, the terms ‘invalid’, referring to incorrect values, and ‘suspicious’, referring to values that are theoretically possible but do not match the validation rules, were used. As illustrated in Figure 5, the diagnostics output identifies six suspicious ‘temp’ values and six invalid ‘humidity’ values, and no invalid values were found in ‘atemp’ and ‘windspeed’.

```
Numerical range diagnostic results:
Invalid humidity values: 6
Invalid windspeed values: 0
Invalid count values: 2
Suspicious temp values: 6
Suspicious atemp values: 0
```

Figure 5. Numerical range diagnostic output before cleaning.

Finally, before cleaning the data, the IQR-based diagnostic check was applied to the ‘count’ variable. The code calculated the first and third quartiles and the interquartile range, then used these values to determine the lower and upper outlier boundaries, as shown in Figure 6. Missing ‘count’ values were excluded from the IQR calculation, as missing target values were handled separately.

```
# Diagnostic check for extreme count outliers using the IQR method

count_data = bike_data["count"].dropna() # Missing count values are ignored for this calculation

Q1 = count_data.quantile(0.25)
Q3 = count_data.quantile(0.75)
IQR = Q3 - Q1

count_lower_bound = max(0, Q1 - 1.5 * IQR)
count_upper_bound = Q3 + 1.5 * IQR

count_outliers = bike_data[
    (bike_data["count"] < count_lower_bound) |
    (bike_data["count"] > count_upper_bound)
]

print("Count outlier diagnostic results:")
print("Count lower bound:", round(count_lower_bound, 2))
print("Count upper bound:", round(count_upper_bound, 2))
print("Number of count outliers identified:", len(count_outliers))

if len(count_outliers) > 0:
    print("\nCount outliers preview:")
    display(count_outliers[["date", "season", "weather", "temp", "humidity", "windspeed", "count"]].head(20))
```

Figure 6. IQR diagnostic code for identifying potential count outliers.

The results of the IQR diagnostic in Figure 7 show the calculated lower (0) and upper (1678.5) bounds and identify six records as potential ‘count’ outliers. These results were later used in the cleaning stage to remove invalid negative rental counts and extremely high rental counts.

Count outlier diagnostic results:							
Count lower bound: 0							
Count upper bound: 1678.5							
Number of count outliers identified: 6							
Count outliers preview:							
	date	season	weather	temp	humidity	windspeed	count
53	22/02/2024	Summer	Cloudy	31.9	77.8	10.0	-50.0
232	12/08/2024	Winter	Rain	17.7	47.4	7.2	50000.0
387	7/01/2025	Summer	Clear	22.1	66.7	9.5	-100.0
454	11/03/2025	Autumn	Clear	27.8	69.9	4.3	20000.0
488	12/04/2025	Autumn	Rain	26.8	69.4	12.3	5000.0
745	16/12/2025	Summer	Clear	NaN	70.5	7.5	10000.0

Figure 7. Output of the IQR diagnostic check.

Data Cleaning and Handling Data-Related Issues

Based on the inspection outputs and validation rules, cleaning was applied to a copy of the original dataset. Duplicate rows were removed to prevent repeated records from distorting descriptive statistics, visualisations and model training. The inconsistent category labels identified earlier were standardised from ‘Summr’ to ‘Summer’ and from ‘Rainn’ to ‘Rain’, as illustrated in Figure 8. Outputs of the standardisation are shown in Figure 9.

```
# Standardise obvious spelling mistakes in categorical values identified during the category check

bike_cleaned["season"] = bike_cleaned["season"].replace({
    "Summr": "Summer"
})

bike_cleaned["weather"] = bike_cleaned["weather"].replace({
    "Rainn": "Rain"
})

print("Season values after standardisation:")
print(bike_cleaned["season"].unique())

print("\nWeather values after standardisation:")
print(bike_cleaned["weather"].unique())
```

Figure 8. Code for Standardisation of inconsistent categorical labels.

```
Season values after standardisation:
<ArrowStringArray>
['Summer', 'Autumn', 'Winter', 'Spring']
Length: 4, dtype: str

Weather values after standardisation:
<ArrowStringArray>
['Cloudy', 'Clear', 'Rain', 'Storm']
Length: 4, dtype: str
```

Figure 9. Output for Standardisation of inconsistent categorical labels.

Missing values were handled according to each variable's role. Hence, rows with missing 'count' values were removed as 'count' was considered the target variable, and replacing it with estimates, whether mean or median, would create artificial rental outcomes. Rows with missing 'temp' values were also removed as temperature was the key predictor variable for both visual analysis and modelling, and imputation could distort the temperature-related relationships. As shown in Figure 10, missing values in 'atemp', 'humidity', and 'windspeed' were filled using the median, since these variables were only explanatory predictors, imputation allowed the records to be retained while reducing the influence of extreme values compared with mean imputation.

```
# Handle missing numerical predictor values
# Rows with missing temp are removed because temp is a key variable for visual analysis and modelling
# Other missing numerical predictors are filled using median imputation

print("Missing values before handling:")
print(bike_cleaned[["temp", "atemp", "humidity", "windspeed"]].isnull().sum())

print("\nShape before removing missing temp values:", bike_cleaned.shape)

# Remove rows with missing temperature
bike_cleaned = bike_cleaned.dropna(subset=["temp"])

print("Shape after removing missing temp values:", bike_cleaned.shape)

# Fill remaining missing numerical predictors with median
numeric_predictors = ["atemp", "humidity", "windspeed"]

for column in numeric_predictors:
    median_value = bike_cleaned[column].median()
    bike_cleaned[column] = bike_cleaned[column].fillna(median_value)

print("\nMissing values after handling:")
print(bike_cleaned[["temp", "atemp", "humidity", "windspeed"]].isnull().sum())
```

Figure 10. Code for handling missing numerical predictor values.

The date column was converted to datetime format using mixed-format parsing and day-first interpretation, and any remaining invalid dates were printed, as shown in Figure 11. The output of the conversion is shown in Figure 12.

```
# Convert date from text to datetime
# Mixed formats possible, so format="mixed" is used

bike_cleaned["date_original"] = bike_cleaned["date"]

bike_cleaned["date"] = pd.to_datetime(
    bike_cleaned["date_original"].astype(str).str.strip(),
    format="mixed",
    dayfirst=True,
    errors="coerce"
)

print("\nDate range:")
print("Minimum date:", bike_cleaned["date"].min())
print("Maximum date:", bike_cleaned["date"].max())

# If any invalid dates remain, print them
invalid_dates = bike_cleaned[bike_cleaned["date"].isnull()]

print("Invalid dates after conversion:", bike_cleaned["date"].isnull().sum())

if len(invalid_dates) > 0:
    print("\nInvalid original date values:")
    display(invalid_dates[["date_original", "season", "weather", "count"]].head(20))
```

Figure 11. Date conversion code.

```
Date range:
Minimum date: 2024-01-01 00:00:00
Maximum date: 2025-12-31 00:00:00
Invalid dates after conversion: 1

Invalid original date values:

```

	date_original	season	weather	count
443	29/02/2025	Summer	Cloudy	1040.0

Figure 12. Date conversion output.

Invalid calendar dates were removed because their correct intended values could not be verified, as shown in Figure 13.

```
# Remove rows where the date could not be converted into a valid datetime

print("Shape before removing invalid dates:", bike_cleaned.shape)
print("Invalid dates before removal:", bike_cleaned["date"].isnull().sum())

bike_cleaned = bike_cleaned.dropna(subset=["date"])

print("\nShape after removing invalid dates:", bike_cleaned.shape)
print("Invalid dates after removal:", bike_cleaned["date"].isnull().sum())
```

Figure 13. Invalid dates removal code.

After invalid dates were removed, out-of-range numerical values were filtered according to the validation rules. Extreme ‘count’ outliers were also removed from ‘count’ using the IQR method, as shown in Figure 14.

```
# Remove extreme outliers in count using the IQR method
print("Shape before count outlier removal:", bike_cleaned.shape)

Q1 = bike_cleaned["count"].quantile(0.25)
Q3 = bike_cleaned["count"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = max(0, Q1 - 1.5 * IQR)
upper_bound = Q3 + 1.5 * IQR

print("Count lower bound:", round(lower_bound, 2))
print("Count upper bound:", round(upper_bound, 2))

bike_cleaned = bike_cleaned[
    (bike_cleaned["count"] >= lower_bound) &
    (bike_cleaned["count"] <= upper_bound)
]

print("Shape after count outlier removal:", bike_cleaned.shape)
print("Maximum count after cleaning:", bike_cleaned["count"].max())
```

Figure 14. IQR outlier removal code.

Numerical values that met the validation rules were retained in their original form rather than rounded, and those that did not meet the validation rules were removed, thereby preserving the dataset’s precision and avoiding small numerical errors that could reduce the reliability of subsequent analysis (Croci et al. 2022).

After all cleaning steps, the dataset was reduced from 761 rows to 671 rows, meaning that 90 rows, or approximately 11.83% of the original data, were removed. The code used to calculate

the reduction and summarise the cleaning is depicted in Figure 15. Although cleaning reduced the dataset size, it helped eliminate records with missing target values, invalid dates, duplicates, unrealistic numerical values, or extreme outliers. Retaining these problematic records could have increased the risk of analysing unreliable data and potentially distorted both the visual analysis and the regression model's training. The summary of cleaning findings is illustrated in Figures 16, 17 and 18.

```
# Summarise the effect of data cleaning on dataset size
original_rows = bike_data.shape[0]
cleaned_rows = bike_cleaned.shape[0]
removed_rows = original_rows - cleaned_rows
removed_percentage = (removed_rows / original_rows) * 100

print("Original number of rows:", original_rows)
print("Final cleaned number of rows:", cleaned_rows)
print("Rows removed during cleaning:", removed_rows)
print("Percentage of rows removed:", round(removed_percentage, 2), "%")

# Final basic cleaning checks to ensure the success of the cleaning
print("\nMissing values after cleaning:")
print(bike_cleaned.isnull().sum())

print("\nDuplicate rows after cleaning:")
print(bike_cleaned.duplicated().sum())
```

Figure 15. Code for the calculation of the reduction and summary of the cleaning process.

```
Original number of rows: 761
Final cleaned number of rows: 671
Rows removed during cleaning: 90
Percentage of rows removed: 11.83 %

Missing values after cleaning:
date          0
season        0
holiday        0
workingday     0
weather        0
temp           0
atemp          0
humidity       0
windspeed     0
count          0
date_original  0
dtype: int64

Duplicate rows after cleaning:
0
```

Figure 16. Cleaning summary results: change in the shape, presence of missing or duplicate records.

```
-Invalid category checks-  
Invalid season values: 0  
Invalid weather values: 0  
  
-Invalid binary checks-  
Invalid holiday values: 0  
Invalid workingday values: 0  
  
Season values after cleaning:  
['Summer', 'Autumn', 'Winter', 'Spring']  
  
Weather values after cleaning:  
['Cloudy', 'Clear', 'Rain', 'Storm']
```

Figure 17. Output for the check of invalid category and binary values.

```
-Invalid date checks-  
Invalid dates: 0  
  
-Invalid numeric range checks-  
Invalid out of range humidity values: 0  
Invalid out of range windspeed values: 0  
Invalid out of range count values: 0  
Invalid out of range temp values: 0  
Invalid out of range atemp values: 0
```

Figure 18. Output for the check of invalid category or out-of-range values.

Descriptive Summary of the Cleaned Dataset

After cleaning, a descriptive summary was created to examine the main numerical, categorical, and binary variables within the cleaned dataset. The code used to generate the numerical summary is shown in Figure 19. As shown in Figure 20, the cleaned dataset contained 671 valid entries for each numerical variable. The average daily rental count was roughly 668.04, and the median was 682, indicating that typical daily demand hovered around 680 rentals. Rental demand represented by the 'count' variable, however, showed significant variation, ranging from 0 to 1522 rentals, with a standard deviation of approximately 314.21, as can be seen in Figure 20. This suggested that bike rental demand fluctuated considerably throughout the dataset period.

```
# Descriptive summary of cleaned numerical variables  
numerical_summary = bike_cleaned[["temp", "atemp", "humidity", "windspeed", "count"]].describe().round(2)  
  
display(numerical_summary)
```

Figure 19. Code for generating the descriptive summary for numerical variables.

	temp	atemp	humidity	windspeed	count
count	671.00	671.00	671.00	671.00	671.00
mean	21.97	22.33	58.02	10.45	668.04
std	5.03	5.16	13.55	3.13	314.21
min	7.40	6.00	20.11	0.80	0.00
25%	18.20	18.54	47.20	8.40	435.00
50%	21.90	22.40	58.12	10.40	682.00
75%	25.90	26.20	67.85	12.50	922.00
max	34.10	37.72	98.40	20.00	1522.00

Figure 20. Descriptive summary output for cleaned numerical variables.

Furthermore, as shown in Figure 20, ‘temp’ ranged from 7.4°C to 34.1°C, with an average of approximately 21.97°C. In comparison, apparent temperature ranged from 6.0°C to 37.72°C. ‘humidity’ varied from approximately 20.11% to 98.4%, and ‘windspeed’ ranged from 0.8 to 20.0 km/h. The values shown in Figure 20 indicate that the cleaned dataset retained a broad range of weather conditions suitable for exploratory analysis and further modelling.

The categorical and binary summaries were also created to examine the distributions of the season, weather, holiday, and working day variables, using the code shown in Figure 21. As can be seen in Figure 22, the season variable was relatively balanced, with each season accounting for about 24-25% of the cleaned dataset. Spring had the highest number of records, with 172 observations, whereas Autumn had the fewest, with 162. Weather conditions were less evenly distributed: clear weather was most frequent with 383 records (57.08%), while storm conditions appeared only 20 times (2.98%). The binary variables revealed that most records were non-holiday days (623 records, or 92.85%) and working days (446 records, or 66.47%). This distribution was important to consider because patterns based on smaller categories, such as storm and holiday observations, may be less stable than patterns based on more common conditions.

```

# Summary of cleaned categorical and binary variables

summary_columns = ["season", "weather", "holiday", "workingday"]

categorical_summary = []

for column in summary_columns:
    counts = bike_cleaned[column].value_counts().reset_index()
    counts.columns = ["Value", "Count"]
    counts["Variable"] = column
    counts["Percentage"] = (counts["Count"] / len(bike_cleaned) * 100).round(2)

    categorical_summary.append(counts)

categorical_summary = pd.concat(categorical_summary, ignore_index=True)

categorical_summary = categorical_summary[[
    "Variable",
    "Value",
    "Count",
    "Percentage"
]]

display(categorical_summary)

```

Figure 21. Code used to generate the categorical and binary variable summary.

	Variable	Value	Count	Percentage
0	season	Spring	172	25.63
1	season	Winter	169	25.19
2	season	Summer	168	25.04
3	season	Autumn	162	24.14
4	weather	Clear	383	57.08
5	weather	Cloudy	199	29.66
6	weather	Rain	69	10.28
7	weather	Storm	20	2.98
8	holiday	0	623	92.85
9	holiday	1	48	7.15
10	workingday	1	446	66.47
11	workingday	0	225	33.53

Figure 22. Categorical and binary variable summary output for the cleaned dataset.

Visual analysis and key insights

The first visualisation analysed seasonal distributions and was created using the code in Figure 23, which grouped the cleaned dataset by season and calculated the mean rental count for each group to create a bar chart, as shown in Figure 24. This visualisation was based on the theory that favourable weather conditions are likely to affect bicycle demand positively. Figure 24

demonstrates a clear seasonal pattern in bike rental demand, supporting this theory. Thus, summer had the highest average rental count, followed by Autumn and Winter, while Spring had the lowest. This suggests that bike rental usage varies meaningfully across the year, rather than remaining stable across seasons. Therefore, the observed results are useful for operational planning as the Council could increase bike availability during higher-demand seasons, particularly Summer and Autumn, and use lower-demand periods for maintenance or targeted promotion.

```
# Calculate average rental count by season
season_avg = bike_cleaned.groupby("season")["count"].mean().sort_values(ascending=True)

season_colours = {
    "Summer": "firebrick",
    "Autumn": "gold",
    "Winter": "steelblue",
    "Spring": "forestgreen"
}

bar_colours = [season_colours[season] for season in season_avg.index]

plt.figure(figsize=(8, 5))
ax = season_avg.plot(kind="barh", color=bar_colours)

plt.title("Average Bike Rentals by Season")
plt.xlabel("Average Rental Count")
plt.ylabel("Season")

plt.tight_layout()
plt.show()
```

Figure 23. Code for Visualisation 1: Average Rentals by Season.

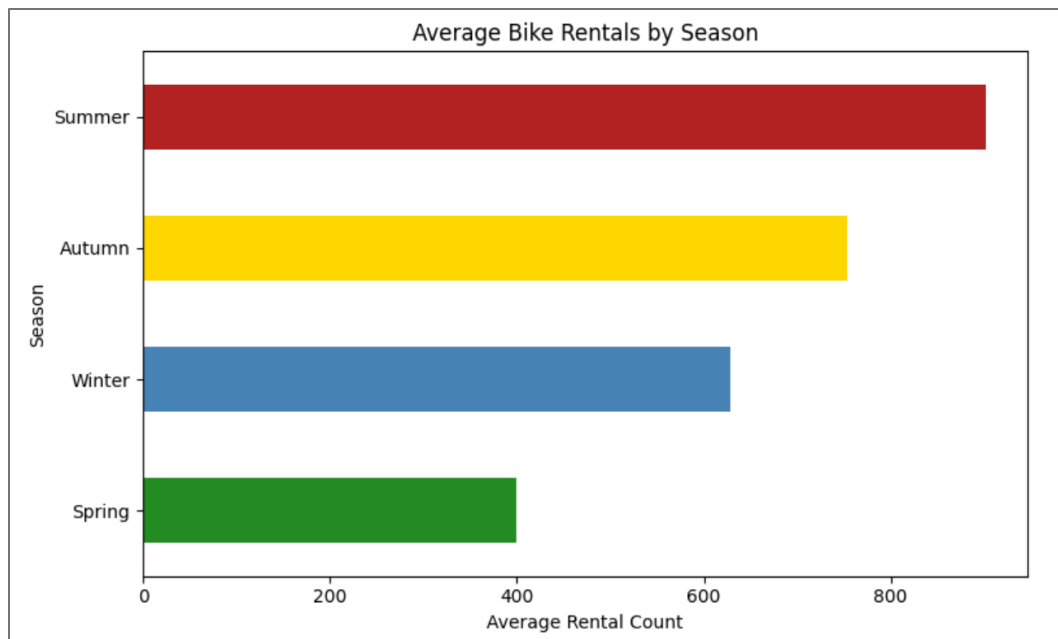


Figure 24. Visualisation 1: Average Rentals by Season.

The second visualisation analysed bike rental distribution by weather conditions using a box plot. This visualisation was created based on the theory that good weather may have increased bike rental demand. The plot was generated using the code in Figure 25, with the final visualisation shown in Figure 26. The created visualisation supports the good-weather theory: clear and cloudy conditions were generally associated with higher rental demand. In contrast, rainy and stormy conditions were associated with lower or more limited usage. However, the overlapping interquartile ranges suggest that weather alone does not fully explain rental demand, as similar rental counts occurred under different weather conditions.

```
# Boxplot of rental count by weather condition
# This shows the distribution and variability of rentals under each weather condition

weather_order = ["Clear", "Cloudy", "Rain", "Storm"]

data_to_plot = [
    bike_cleaned[bike_cleaned["weather"] == weather]["count"]
    for weather in weather_order
]

plt.figure(figsize=(9, 5))
plt.boxplot(data_to_plot, tick_labels=weather_order)

plt.title("Bike Rental Distribution by Weather Condition")
plt.xlabel("Weather Condition")
plt.ylabel("Rental Count")
plt.tight_layout()
plt.show()
```

Figure 25. Code for Visualisation 2: Bike Rental Distribution by Weather Condition.

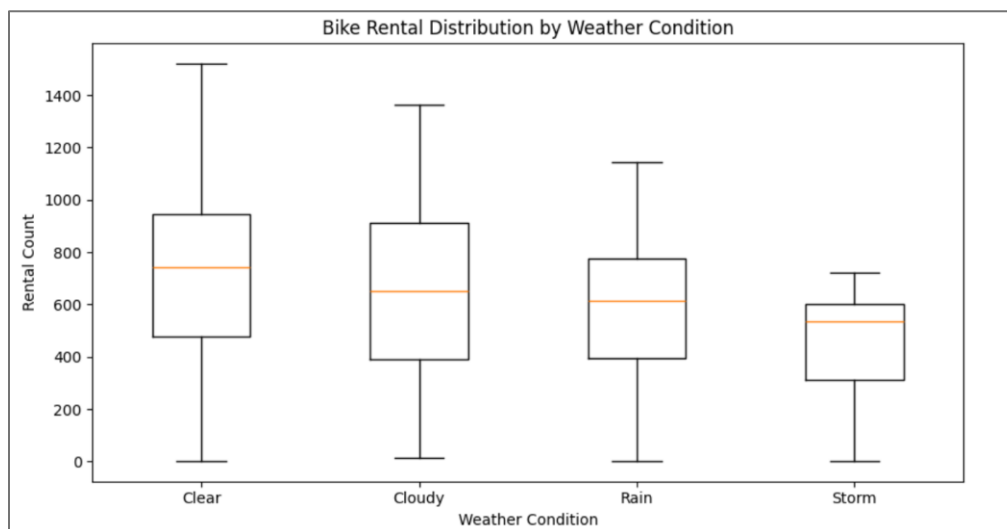


Figure 26. Visualisation 2: Bike Rental Distribution by Weather Condition.

The third visualisation examined the relationship between temperature and bike rental demand across seasons as a scatterplot. The code in Figure 27 creates a graph using three attributes: 'temp', 'count', and 'season', and the resulting scatter plot is shown in Figure 28. A scatterplot was chosen for the three-attribute visualisation to explore the theory that the combined influence of temperature and season affected bike rental demand. It was believed that temperature alone

did not fully explain rental behaviour, as the same temperature could have different effects depending on the season. Visualisation 3 suggests that rental demand generally increased as temperature moved into a more comfortable range, particularly around 22-26°C. However, the relationship was not strictly linear: higher temperatures did not always result in higher rental counts, as shown in Figure 28. The seasonal colouring suggests that Summer had the strongest rental demand, while Spring generally had lower rental counts. This indicates that temperature significantly influences bike demand, but its impact should be considered alongside seasonal factors rather than in isolation. The visualisation is insightful as it reveals a more complex relationship between temperature, season, and rental demand than the simple bar chart in Figure 24 did, allowing the Council to make more context-aware planning decisions about bike availability, maintenance timing and seasonal demand management.

```
# Three attributes: temp, count, and season

season_colours = {
    "Summer": "firebrick",
    "Autumn": "gold",
    "Winter": "steelblue",
    "Spring": "forestgreen"
}

plt.figure(figsize=(9, 6))

for season, colour in season_colours.items():
    season_data = bike_cleaned[bike_cleaned["season"] == season]
    plt.scatter(
        season_data["temp"],
        season_data["count"],
        label=season,
        alpha=0.7,
        color=colour
    )

plt.title("Temperature vs Bike Rentals by Season")
plt.xlabel("Temperature (°C)")
plt.ylabel("Rental Count")
plt.legend(title="Season")
plt.tight_layout()
plt.show()
```

Figure 27. Code for Visualisation 3: Temperature vs Rentals by Season.

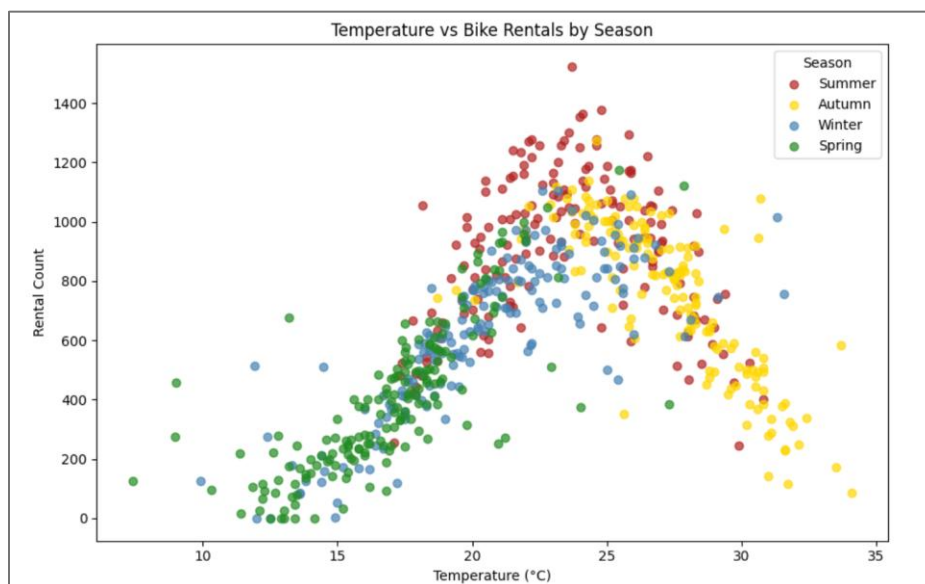


Figure 28. Visualisation 3: Temperature vs Rentals by Season.

The fourth visualisation showed how demand for bicycle sharing changes over time. The code in Figure 29 sorts the dataset by date and displays daily rental count across the full dataset period, while Figure 30 shows the visualisation generated by the code. This time-series visualisation was chosen to test whether cyclical patterns would be exhibited in bike rental demand rather than stability over the dataset period. To reveal broader patterns beyond individual daily fluctuations, demand was plotted over time, as daily rentals may rise and fall with the seasons, weather, holidays, and changing routines. The visualisation allows for identifying these changes and helps demonstrate the benefits of demand forecasting for operational planning. For example, identifying periods of high and low demand can help the council prepare for future decisions about bike availability, staffing, maintenance, and station expansion. Figure 30 also shows that bike rental demand fluctuated strongly from day to day, but the 30-day rolling average revealed a clearer seasonal pattern. Demand declined from September to October in both 2024 and 2025, then increased sharply toward the end of each year and into the summer. This supports the earlier seasonal findings, in which Summer had the highest average rental demand and Spring the lowest. The visualisation is useful because it shows not only daily variation but also broader demand cycles over time, helping the Council plan bike availability, maintenance and staffing around predictable high- and low-demand periods.

```
# Rental demand over time with rolling average
# Rolling average helps show the general trend more clearly

bike_time = bike_cleaned.sort_values("date").copy()

# Calculate 30-day rolling average
bike_time["rolling_30_day_avg"] = bike_time["count"].rolling(window=30).mean()

plt.figure(figsize=(10, 5))

# Daily demand
plt.plot(
    bike_time["date"],
    bike_time["count"],
    color="lightsteelblue",
    alpha=0.5,
    label="Daily rental count"
)

# Smoothed trend
plt.plot(
    bike_time["date"],
    bike_time["rolling_30_day_avg"],
    color="navy",
    linewidth=2.5,
    label="30-day rolling average"
)

plt.title("Bike Rental Demand Over Time")
plt.xlabel("Date")
plt.ylabel("Rental Count")
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```

Figure 29. Code for Visualisation 4: Bike Rental Demand Over Time.

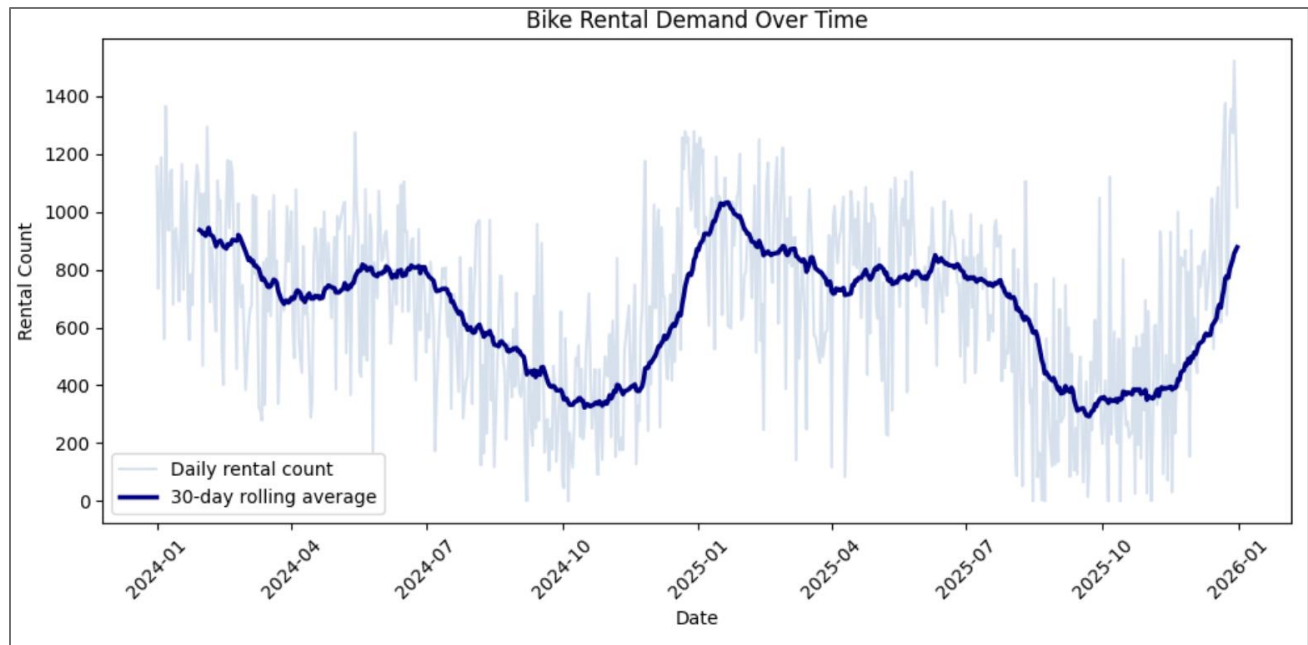


Figure 30. Visualisation 4: Bike Rental Demand Over Time.

Overall, the cleaned dataset showed that bike rental demand varied, with summer being the highest and spring being the lowest, highlighting seasonal influence as illustrated in Figure 24. Weather also affected demand, with clear days accounting for 57.08% and storms only 2.98%, as shown in Figure 22. This distribution suggests that weather affects demand, but it should be interpreted with caution for rare events such as storms. The temperature scatterplot in Figure 28 indicated stronger demand between 22 and 26 °C. The time series in Figure 30 revealed a dip in demand from September to October 2024 and 2025, with increases towards year-end and in summer. These insights show that demand is influenced by seasonality, weather, and temperature, aiding predictive modelling and planning for bike availability, maintenance, and station capacity.

Subtask 2: Prediction Model Discussion

Model preparation and 70/30 Train-Test Split

The cleaned dataset was used to build and evaluate regression models for predicting daily bike rentals, represented by the target variable 'count'. A separate modelling DataFrame was created from the cleaned dataset to complete feature preparation without altering the dataset used earlier for descriptive and visual analysis. As shown in Figure 31, the target variable 'y' was set as 'count', while the feature set 'X' was created by removing 'count', 'date' and 'date_original'. The 'count' column was removed from 'X' because it was the value the model was trying to predict rather than an explanatory feature. The raw date column was excluded because it could not be used directly as a model input. At the same time, 'date_original' was removed because it was merely a textual representation of the same date information. The 'month' feature was extracted because scikit-learn guidance on time-related feature engineering shows that raw date

information should be transformed into usable time-based predictors for regression models (scikit-learn developers n.d.). For this dataset, ‘month’ preserved seasonal information that could help predict bike rental demand.

```
# Prepare modelling dataset
model_data = bike_cleaned.copy()

# Create a simple date-based feature as it may help the model capture seasonal/time-based demand patterns
model_data["month"] = model_data["date"].dt.month

# Target variable
y = model_data["count"]

# Features: remove target and non-model columns
# date is removed because raw datetime cannot be used directly in this model
# date_original is removed because it is a text version of the date
X = model_data.drop(columns=["count", "date", "date_original"])

# Convert categorical variables into dummy variables
X = pd.get_dummies(
    X,
    columns=["season", "weather"],
    drop_first=True
)
```

Figure 31. Code for feature engineering and target-feature setup.

The categorical variables season and weather were then converted into dummy variables, as shown in Figure 32, because regression models require categorical predictors to be represented numerically. Dummy coding converts categorical variables into binary 0/1 variables, allowing each category to be included as a separate predictor in the model (UCLA Statistical Consulting Group n.d.). The output in Figure 33 confirmed that this transformation produced a fully numeric modelling dataset, with 671 records, 13 predictor variables, no missing values in ‘X’ or ‘y’, and no non-numeric columns remaining.

```
# Convert categorical variables into dummy variables
X = pd.get_dummies(
    X,
    columns=["season", "weather"],
    drop_first=True
)

# Convert boolean columns to integers for cleaner modelling output
bool_columns = X.select_dtypes(include=["bool"]).columns
X[bool_columns] = X[bool_columns].astype(int)
```

Figure 32. Code for dummy-variable conversion and modelling readiness checks.

Feature columns:													
Index(['holiday', 'workingday', 'temp', 'atemp', 'humidity', 'windspeed', 'month', 'season_Spring', 'season_Summer', 'season_Winter', 'weather_Cloudy', 'weather_Rain', 'weather_Storm'], dtype='str')													
Non-numeric columns in X:													
[]													
Missing values in X: 0													
Missing values in y: 0													
Feature dataset shape: (671, 13)													
Target shape: (671,)													
	holiday	workingday	temp	atemp	humidity	windspeed	month	season_Spring	season_Summer	season_Winter	weather_Cloudy	weather_Rain	weather_Storm
0	1	0	21.5	21.6	68.8	11.9	1	0	1	0	1	0	0
1	1	0	19.6	18.7	60.1	11.8	1	0	1	0	0	0	0
2	1	0	22.2	24.3	84.5	15.7	1	0	1	0	0	1	0
3	1	0	25.0	22.4	73.1	18.4	1	0	1	0	1	0	0
4	1	0	20.1	20.5	66.4	4.9	1	0	1	0	0	0	0

Figure 33. Output of dummy-variable conversion and modelling readiness checks.

Subsequently, the dataset was split into 70% training data and 30% testing data using 'train_test_split', with the code shown in Figure 34 and the output shown in Figure 35. This produced 469 training records and 202 testing records, allowing the models to be fitted on the training data and evaluated on unseen test data.

```
# Split data into 70% training and 30% testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

Figure 34. Code for the 70/30 train-test split.

```
Training set size: (469, 13)
Testing set size: (202, 13)
```

Figure 35. Output for 70/30 train-test split.

Regression Models Development

Although the task required developing a single regression model, two regression models were trained to facilitate later reflection on the trade-off between accuracy and interpretability. The Random Forest Regressor, depicted in Figure 36, was chosen as the primary predictive model because the exploratory analysis indicated that bike rental demand was not entirely linear. As

shown in the scatterplot in Figure 28, rental demand generally increased within comfortable temperature ranges but did not continue to rise at higher temperatures. This made the Random Forest an appropriate model to test, as it combines multiple decision-tree predictors and can better capture complex relationships than a simple linear model (Breiman 2001). Linear Regression, shown in Figure 37, was used as a simpler baseline model because it provided a clearer, easier-to-interpret comparison. The final model choice was not assumed at this stage; it was confirmed later using the test-set evaluation metrics.

```
# Train Random Forest model
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

# Make predictions on the unseen test set
rf_predictions = rf_model.predict(X_test)

# Create a small comparison table of actual and predicted values
rf_results = pd.DataFrame({
    "Actual count": y_test.values,
    "Predicted count": rf_predictions
})

# Round predicted values for cleaner display
rf_results["Predicted count"] = rf_results["Predicted count"].round(2)

print("Random Forest model trained successfully.")
print("Number of test predictions made:", len(rf_predictions))

rf_results.head(10)
```

Figure 36. Random Forest Model code.

```
# Train Linear Regression model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Make predictions on the unseen test set
linear_predictions = linear_model.predict(X_test)

# Create a small comparison table of actual and predicted values
linear_results = pd.DataFrame({
    "Actual count": y_test.values,
    "Predicted count": linear_predictions
})

# Round predicted values for cleaner display
linear_results["Predicted count"] = linear_results["Predicted count"].round(2)

print("Linear Regression model trained successfully.")
print("Number of test predictions made:", len(linear_predictions))

linear_results.head(10)
```

Figure 37. Linear regression model code.

Feature importance

Feature importance was calculated from the Random Forest model to identify which predictors contributed most strongly to the prediction, as shown in Figure 38. The feature importance table in Figure 39 showed that temp was the dominant predictor, with an importance score of approximately 0.745. This meant that ‘temp’ contributed most strongly to the Random Forest model’s predictions. The next most important predictors were ‘atemp’ at approximately 0.099,

followed by ‘humidity’, ‘windspeed’ and ‘month’, although their importance scores were much smaller.

```
# Random Forest feature importance
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
})

feature_importance = feature_importance.sort_values(by="Importance", ascending=False)

display(feature_importance)
```

Figure 38. Code for Random Forest feature importance table.

	Feature	Importance
2	temp	0.745116
3	atemp	0.099480
4	humidity	0.030049
5	windspeed	0.021114
6	month	0.020425
12	weather_Storm	0.016432
11	weather_Rain	0.015425
1	workingday	0.014472
8	season_Summer	0.014342
0	holiday	0.011576
9	season_Winter	0.004465
10	weather_Cloudy	0.003881
7	season_Spring	0.003224

Figure 39. Random Forest feature importance table output.

The top-10 feature-importance plot in Figure 40 visualises this ranking and makes the dominance of temp especially clear. This supports the earlier visual analysis, where temperature and season were shown to be closely related to rental demand. Feature importance shows which variables the Random Forest relied on most when making predictions. However, it should not be read as proof that those variables directly caused changes in bike rental demand.

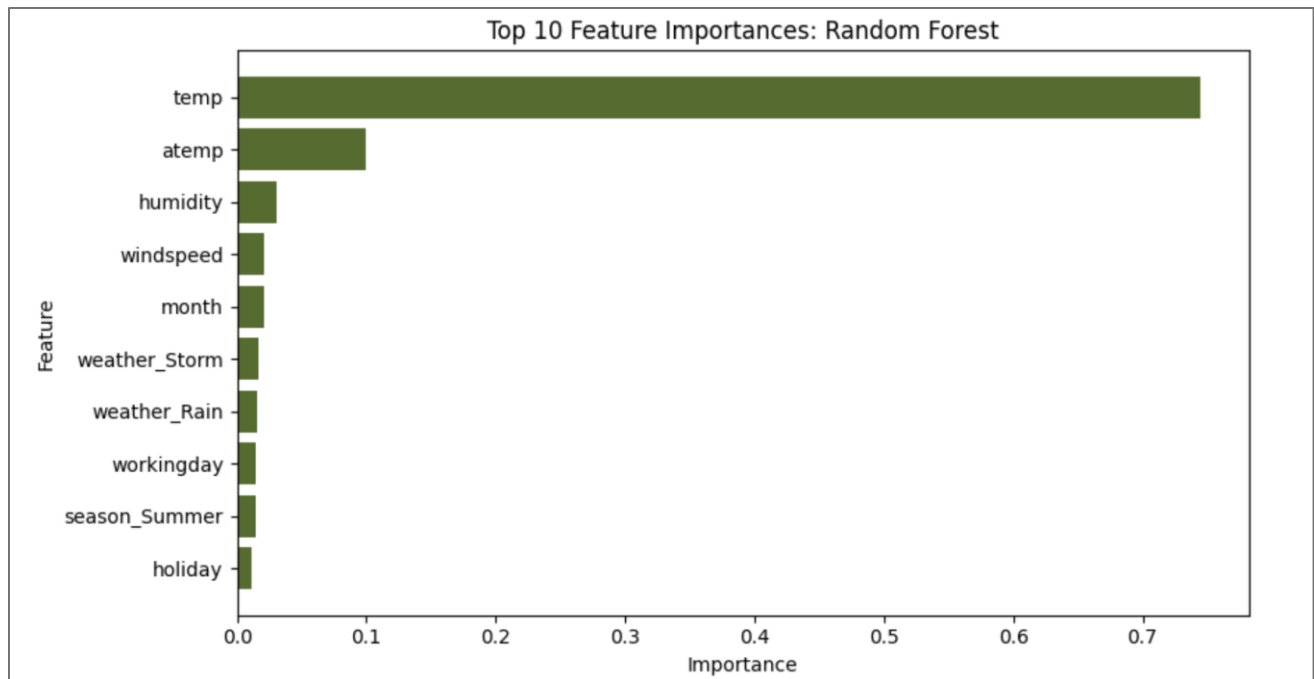


Figure 40. Top 10 features importance plot for Random Forest.

Model Evaluation and Metric Justification

Both models developed were evaluated on the testing set using RMSE, MAE and R^2 . RMSE and MAE measure prediction error in rental-count units, while R^2 indicates how much variation in rental demand the model explains. The code created for evaluation is shown in Figures 41 and 42. The Random Forest model performed better across all metrics, confirming an earlier assumption that it would be better suited to capturing the nonlinear patterns observed in the visual analysis. It achieved an RMSE of 113.60, MAE of 87.57 and R^2 of 0.871, compared with the Linear Regression model's RMSE of 229.47, MAE of 185.09 and R^2 of 0.473, as shown in Figure 43.

```
# Evaluate Random Forest using RMSE, MAE and R2
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_predictions))
rf_mae = mean_absolute_error(y_test, rf_predictions)
rf_r2 = r2_score(y_test, rf_predictions)

# Evaluate Linear Regression using RMSE, MAE and R2
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_predictions))
linear_mae = mean_absolute_error(y_test, linear_predictions)
linear_r2 = r2_score(y_test, linear_predictions)
```

Figure 41. Code used for evaluation of Baseline and Main models.

```

# Compare model performance using regression metrics
# R² (%) shows R² expressed as a percentage for easier interpretation
# The best model is selected based on the lowest RMSE

model_results = pd.DataFrame({
    "Model": ["Linear Regression", "Random Forest"],
    "RMSE": [linear_rmse, rf_rmse],
    "MAE": [linear_mae, rf_mae],
    "R²": [linear_r2, rf_r2],
    "R² (%)": [linear_r2 * 100, rf_r2 * 100]
})

model_results["RMSE"] = model_results["RMSE"].round(2)
model_results["MAE"] = model_results["MAE"].round(2)
model_results["R²"] = model_results["R²"].round(3)
model_results["R² (%)"] = model_results["R² (%)"].round(2)

display(model_results)

best_model = model_results.sort_values(by="RMSE").iloc[0]

print("Best performing model based on RMSE:")
print("Model:", best_model["Model"])
print("RMSE:", best_model["RMSE"])
print("MAE:", best_model["MAE"])
print("R²:", best_model["R²"])
print("R² (%):", best_model["R² (%)"])

```

Figure 42. Code used to compare Baseline and Main models.

	Model	RMSE	MAE	R²	R² (%)
0	Linear Regression	229.47	185.09	0.473	47.26
1	Random Forest	113.60	87.57	0.871	87.07
Best performing model based on RMSE:					
Model: Random Forest					
RMSE: 113.6					
MAE: 87.57					
R²: 0.871					
R² (%): 87.07					

Figure 43. Models' evaluation output.

RMSE was considered the most meaningful metric in this context because it is measured in the same units as the target variable 'count' and penalises larger prediction errors more strongly. This can be useful for future Council planning, as it highlights the kinds of larger-scale forecasting errors that could affect bike availability, maintenance timing, and station capacity.

Accuracy and Interpretability Trade-Off

The results show a clear trade-off between accuracy and interpretability. Linear Regression is easier to interpret because it uses a simpler linear structure, making it more transparent how predictors relate to rental 'count'. This aligns with the idea that linear regression is inherently interpretable, whereas more complex models, such as Random Forest, are usually less transparent and may require post hoc interpretation methods (Molnar n.d.). However, Linear Regression performed less accurately, with an RMSE of 229.47 and an R^2 of 0.473. Random Forest was less directly interpretable because it combines many decision trees, but it achieved much stronger predictive performance, with an RMSE of 113.60 and an R^2 of 0.871, as shown in Figure 43. In this context, Random Forest was the better model for forecasting because the task focused on accurately predicting bike rental demand. However, Linear Regression remained useful as a baseline model because it provided a simpler point of comparison and helped demonstrate the improvement achieved by using a more flexible model.

Conclusion and Recommendations

Main Findings

The analysis showed that bike rental demand was strongly shaped by environmental and seasonal factors. After cleaning, the dataset contained 671 valid daily records, and the visual analysis demonstrated clear demand patterns across season, weather, temperature and time. Summer had the highest average rental demand, while Spring had the lowest average demand among the four seasons. The modelling results also supported these patterns: the Random Forest model performed much better than Linear Regression, achieving an RMSE of 113.60, an MAE of 87.57, and an R^2 of 0.871. This means the model explained approximately 87.07% of the variation in rental count. Feature importance showed that 'temp' was the strongest predictor, followed by 'atemp', 'humidity', 'windspeed', and 'month'.

Limitations, Biases, and Overfitting Risks

Although the Random Forest model performed well, several limitations should be considered. First, the cleaned dataset was reduced from 761 to 671 records, meaning that about 12% of the information was lost during cleaning. This was necessary to remove unreliable records, but it may still reduce the dataset's coverage of unusual demand conditions. Second, some categories were unevenly represented: for example, clear weather appeared much more often than storm conditions, meaning the model may be less reliable for rare weather scenarios. Third, Random Forest can capture complex non-linear patterns, but it also carries a risk of overfitting, especially with a relatively small dataset. The use of a separate 30% test set helped reduce this risk, but further validation would be needed before using the model for real operational decisions.

Recommendations for future improvement

Future work should use a larger, more recent dataset, ideally covering additional years, locations, and station-level rental activity. This would help the model learn more stable patterns across rare

events, peak periods and unusual weather conditions. Additional features could also improve forecasting accuracy, such as public holidays, school holidays, rainfall amount, local events and station location. The model could be further improved by tuning Random Forest hyperparameters and using cross-validation to test whether performance remains stable across different data splits. For Council planning, the model should be updated regularly so that predictions continue to reflect changes in travel behaviour, seasonal demand, and local infrastructure use.

References

- Breiman, L 2001, 'Random forests', *Machine Learning*, vol. 45, no. 1, pp. 5–32, viewed 29 May 2026, <https://doi.org/10.1023/A:1010933404324>
- Bureau of Meteorology n.d., Climate statistics for Australian locations: Rockhampton Aero, *Bureau of Meteorology*, viewed 15 May 2026, https://www.bom.gov.au/climate/averages/tables/cw_039083_All.shtml
- Croci, M, Fasi, M, Higham, NJ, Mary, T & Mikaitis, M 2022, 'Stochastic rounding: implementation, error analysis and applications', *Royal Society Open Science*, vol. 9, no. 3, article 211631, viewed 23 May 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC8905452/>
- Current Results n.d., Rockhampton extreme annual low temperatures, *Current Results*, viewed 20 May 2026, <https://www.currentresults.com/Yearly-Weather/Australia/QLD/Rockhampton/extreme-annual-rockhampton-low-temperature.php>
- Molnar, C n.d., *Interpretable machine learning*, viewed 27 May 2026, <https://christophm.github.io/interpretable-ml-book/overview.html>
- scikit-learn developers n.d., Time-related feature engineering, *scikit-learn*, viewed 26 May 2026, https://scikit-learn.org/stable/auto_examples/applications/plot_cyclical_feature_engineering.html
- UCLA Statistical Consulting Group n.d., Coding systems for categorical variables in regression analysis, *UCLA Advanced Research Computing*, viewed 27 May 2026, <https://stats.oarc.ucla.edu/spss/faq/coding-systems-for-categorical-variables-in-regression-analysis-2/>